Medcare- Pharmacy Management Application

Technical Documentation

Project Documentation

February 13, 2026

Contents

1 Project Overview **3**
- 1.1 Key Features . . . 3
- 1.2 Architecture . . . 3

2 System Diagrams **4**
- 2.1 Use Case Diagram . . . 4
- 2.2 Data Flow Diagram (DFD) - Level 0 . . . 5
- 2.3 Data Flow Diagram (DFD) - Level 1 . . . 5
- 2.4 Entity-Relationship (ER) Diagram . . . 6
- 2.5 Project Timeline - Gantt Chart . . . 7
- 2.6 System Architecture Block Diagram . . . 8

3 Languages and Technologies Used **9**
- 3.1 Programming Languages . . . 9
- 3.2 Backend Packages (Node.js) . . . 9
- 3.3 Frontend Packages (React.js) . . . 10

4 Database Schema **11**
- 4.1 User Collection . . . 11
- 4.2 Product Collection . . . 11
- 4.3 Order Collection . . . 11
- 4.4 Contact Collection . . . 11

5 Step-wise Algorithm **12**
- 5.1 User Registration Process . . . 12
- 5.2 User Login Process . . . 13
- 5.3 Product Browsing Process . . . 13
- 5.4 Add to Cart Process . . . 13
- 5.5 Checkout and Payment Process . . . 14
- 5.6 Order Management Process . . . 14
- 5.7 Contact Form Submission . . . 15

6 API Endpoints **16**

7 System Requirements **17**
7.1 Software Requirements 17
7.2 Environment Variables 17

8 Conclusion **17**

1 Project Overview

The **Medcare-Pharmacy Management Application** is a full-stack web application designed to facilitate online pharmacy operations. It provides a platform for customers to browse, search, and purchase pharmaceutical products online with secure payment processing.

1.1 Key Features

- User registration and authentication with JWT tokens
- Product catalog with categories (medicines, healthcare products)
- Shopping cart functionality with quantity management
- Secure payment integration via Stripe
- Order management and tracking
- Contact form for customer inquiries
- Prescription upload functionality
- Admin capabilities for product and order management
- Responsive design for mobile and desktop

1.2 Architecture

The application follows a **MERN Stack** architecture:

- **MongoDB** – NoSQL database for data storage
- **Express.js** – Backend web framework
- **React.js** – Frontend library for UI
- **Node.js** – Server-side JavaScript runtime

2 System Diagrams

2.1 Use Case Diagram

The Use Case diagram illustrates the interactions between actors (Customer and Admin) and the system functionalities.

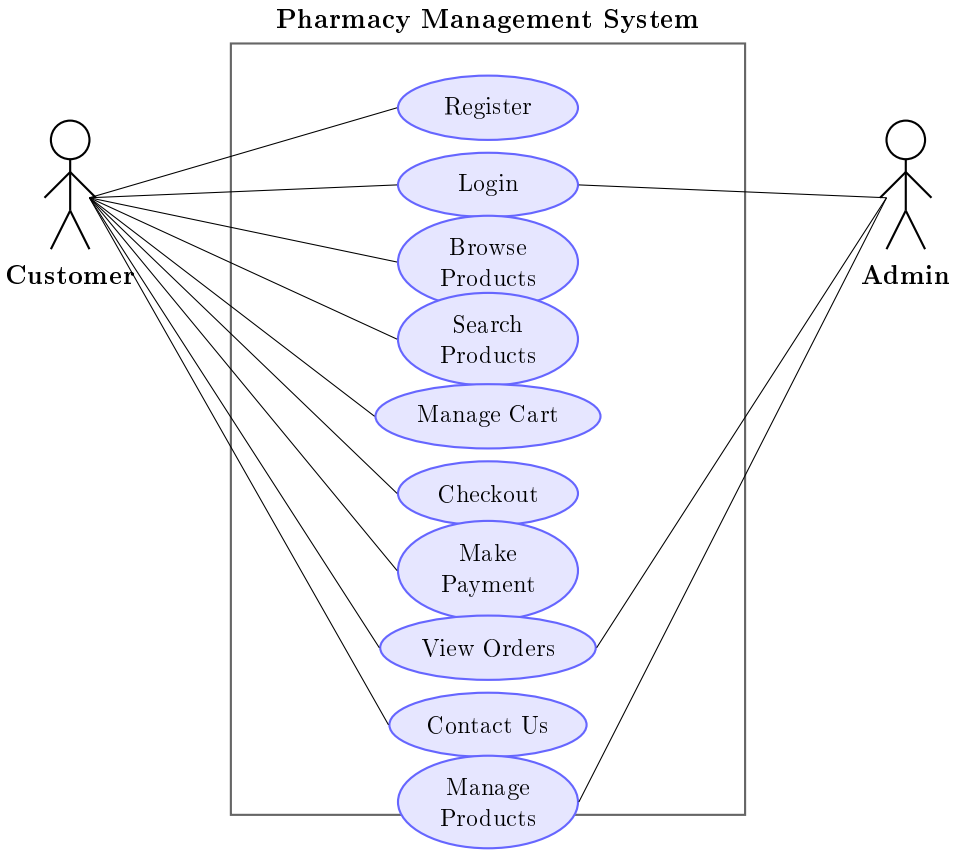

Figure 1: Use Case Diagram

2.2 Data Flow Diagram (DFD) - Level 0

The Context Diagram (Level 0 DFD) shows the system as a single process with external entities.

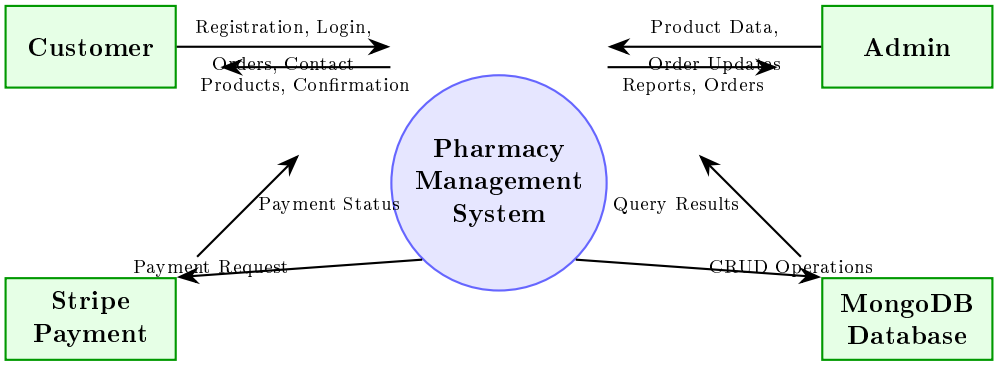

Figure 2: Context Diagram (Level 0 DFD)

2.3 Data Flow Diagram (DFD) - Level 1

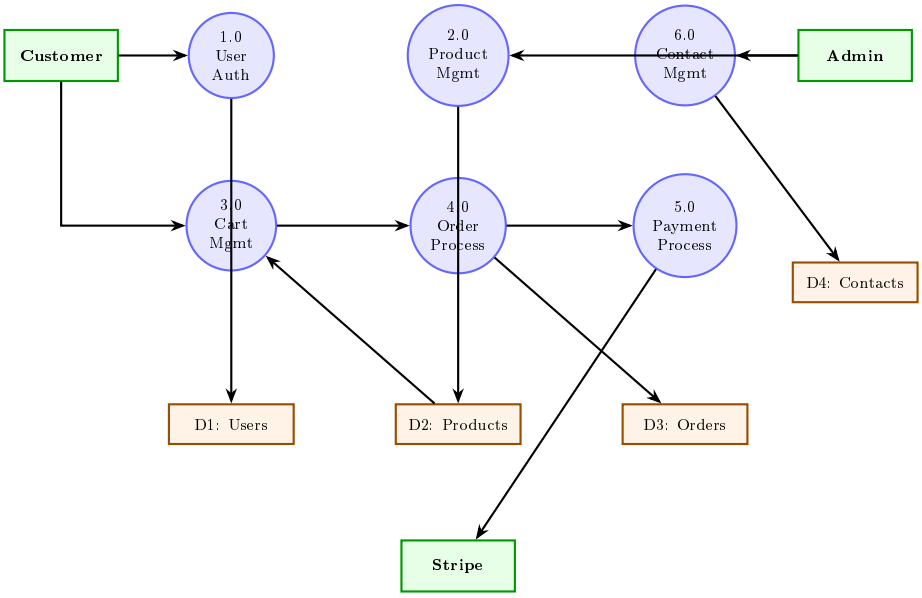

Figure 3: Level 1 Data Flow Diagram

2.4 Entity-Relationship (ER) Diagram

The ER diagram shows the database structure and relationships between entities.

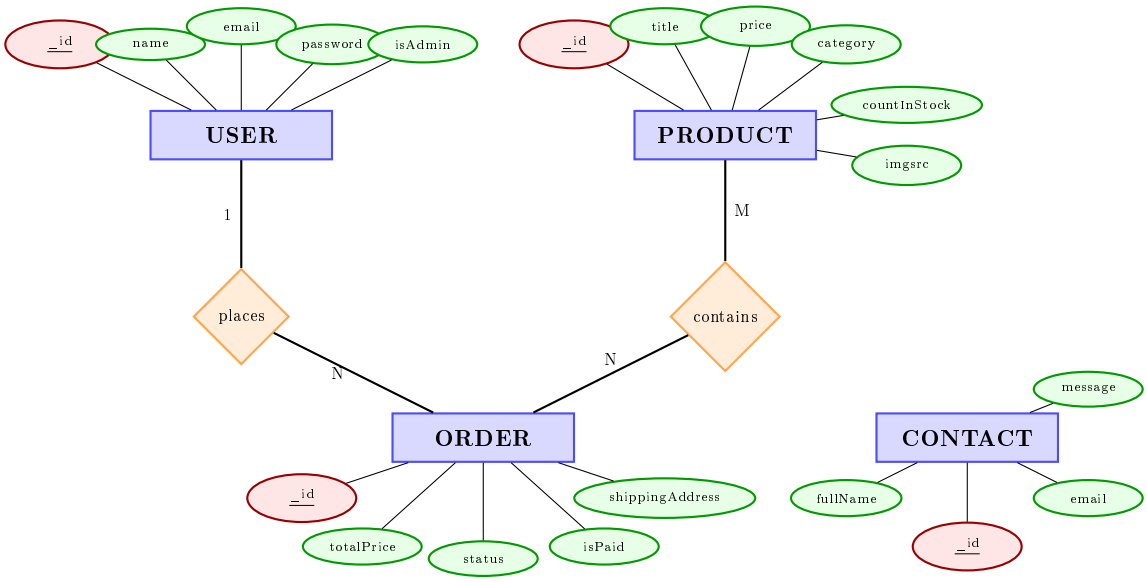

Figure 4: Entity-Relationship Diagram

2.5 Project Timeline - Gantt Chart

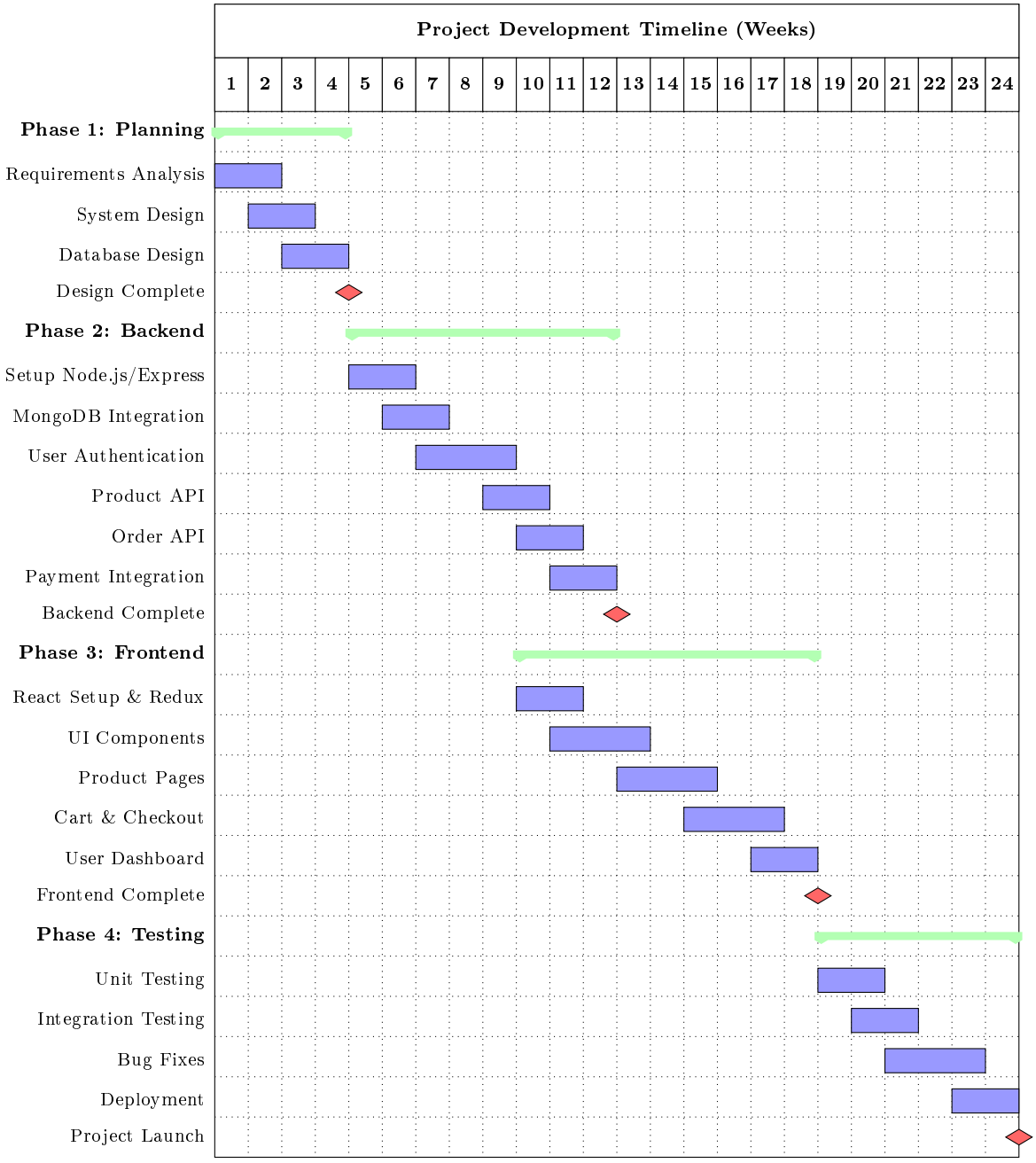

Figure 5: Project Development Gantt Chart

2.6 System Architecture Block Diagram

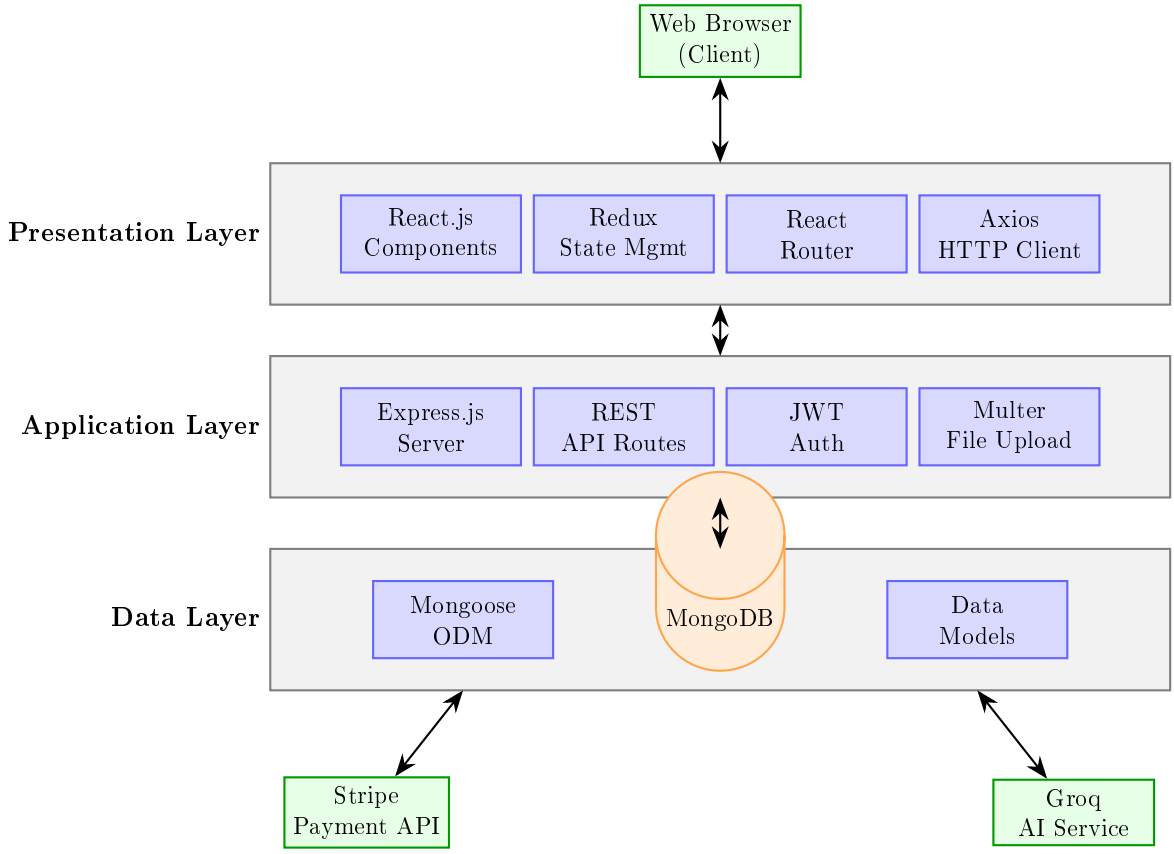

Figure 6: System Architecture Block Diagram

3 Languages and Technologies Used

3.1 Programming Languages

Language	Version	Purpose
JavaScript (ES6+)	ECMAScript 2020	Primary language for both frontend and backend
HTML5	5	Markup language for web pages
CSS3	3	Styling and layout of web pages

Table 1: Programming Languages

3.2 Backend Packages (Node.js)

Package	Version	Purpose
express	4.17.1	Web application framework for creating REST APIs and handling HTTP requests
mongoose	5.11.8	MongoDB object modeling tool for data validation and schema management
bcryptjs	2.4.3	Password hashing library for secure user authentication
jsonwebtoken	8.5.1	JWT implementation for user authentication and session management
cors	2.8.5	Middleware to enable Cross-Origin Resource Sharing between frontend and backend
dotenv	8.2.0	Loads environment variables from .env file for configuration
stripe	8.150.0	Payment processing integration for handling online transactions
multer	2.0.2	Middleware for handling multipart/form-data for file uploads
body-parser	1.19.0	Parses incoming request bodies (JSON, URL-encoded)
uuid	8.3.2	Generates unique identifiers for products and orders
groq-sdk	0.37.0	AI/LLM integration for advanced features
nodemon	2.0.6	Development tool for auto-restarting server on file changes
concurrently	5.3.0	Runs multiple npm scripts simultaneously during development

Package	Version	Purpose

Table 2: Backend Dependencies

3.3 Frontend Packages (React.js)

Package	Version	Purpose
react	19.2.4	Core library for building user interfaces with components
react-dom	19.2.4	React package for DOM manipulation and rendering
react-router-dom	5.2.0	Declarative routing for navigation between pages
redux	4.0.5	State management library for predictable state container
react-redux	7.2.2	Official React bindings for Redux state management
redux-thunk	2.3.0	Middleware for async actions in Redux (API calls)
redux-devtools-extension	2.13.8	Browser extension support for Redux debugging
axios	0.21.1	HTTP client for making API requests to backend
react-stripe-checkout	2.6.3	React component for Stripe payment checkout
framer-motion	11.11.17	Animation library for smooth UI transitions
gsap	3.14.2	Professional-grade animation library for complex animations
react-scripts	4.0.1	Create React App build scripts and configuration
@testing-library/react	11.2.2	Testing utilities for React components
@testing-library/jest-dom	5.11.6	Custom Jest matchers for DOM testing

Table 3: Frontend Dependencies

4 Database Schema

The application uses **MongoDB** as the database with **Mongoose ODM** for schema definition. Below are the database collections and their schemas.

4.1 User Collection

Field	Type	Required	Description
_id	ObjectId	Auto	Unique identifier (MongoDB generated)
name	String	Yes	Full name of the user
email	String	Yes	User email (unique)
password	String	Yes	Hashed password (bcrypt)
isAdmin	Boolean	Yes	Admin privilege flag (default: false)
createdAt	Date	Auto	Timestamp of creation
updatedAt	Date	Auto	Timestamp of last update

Table 4: User Schema

4.2 Product Collection

Field	Type	Required	Description
_id	ObjectId	Auto	Unique identifier (MongoDB generated)
unique_id	String	Yes	Custom unique product ID
imgsrc	String	Yes	URL/path to product image
title	String	Yes	Product name/title
price	Number	Yes	Product price
category	String	Yes	Product category
indication	String	No	Medical indication/use
dosage	String	No	Recommended dosage
sideEffects	String	No	Known side effects
countInStock	Number	No	Available quantity (default: 0)

Table 5: Product Schema

4.3 Order Collection

Order Status Values: Pending, Processing, Shipped, Delivered, Cancelled

4.4 Contact Collection

Field	Type	Required	Description
_id	ObjectId	Auto	Unique identifier
user	ObjectId	Yes	Reference to User collection
orderItems	Array	Yes	Array of ordered products
shippingAddress	Object	Yes	Delivery address details
paymentMethod	String	Yes	Payment method used
paymentResult	Object	No	Payment transaction details
taxPrice	Number	Yes	Tax amount (default: 0.0)
shippingPrice	Number	Yes	Shipping cost (default: 0.0)
totalPrice	Number	Yes	Total order amount
isPaid	Boolean	Yes	Payment status (default: false)
paidAt	Date	No	Payment timestamp
status	String	Yes	Order status (enum)
isDelivered	Boolean	Yes	Delivery status (default: false)
deliveredAt	Date	No	Delivery timestamp
createdAt	Date	Auto	Order creation timestamp
updatedAt	Date	Auto	Last update timestamp

Table 6: Order Schema

Field	Type	Required	Description
_id	ObjectId	Auto	Unique identifier
fullName	String	No	Name of the contact person
email	String	No	Contact email address
gender	String	No	Gender of the person
message	String	No	Message/inquiry content
city	String	No	City of the contact

Table 7: Contact Schema

5 Step-wise Algorithm

5.1 User Registration Process

Step 1: User fills out the registration form with name, email, and password

Step 2: Frontend validates the input fields (non-empty, valid email format)

Step 3: Frontend sends POST request to `/api/users/register`

Step 4: Backend checks if email already exists in the database

Step 5: If email exists, return error "User already exists"

Step 6: If new user, hash the password using bcrypt (10 salt rounds)

Step 7: Create new user document in MongoDB

Step 8: Generate JWT token with user ID

Step 9: Return user data and token to frontend

Step 10: Frontend stores token in localStorage and redirects to home

5.2 User Login Process

Step 1: User enters email and password in login form

Step 2: Frontend sends POST request to `/api/users/login`

Step 3: Backend finds user by email in database

Step 4: If user not found, return error "Invalid credentials"

Step 5: Compare entered password with stored hash using bcrypt

Step 6: If password matches, generate JWT token

Step 7: Return user data and token to frontend

Step 8: Frontend stores token and updates Redux state

Step 9: User is redirected to their intended page

5.3 Product Browsing Process

Step 1: User opens the application home page

Step 2: Frontend dispatches action to fetch products

Step 3: GET request sent to `/api/products`

Step 4: Backend queries MongoDB for all products

Step 5: Products are returned as JSON array

Step 6: Redux store updates with product list

Step 7: React components render product cards

Step 8: User can filter products by category

Step 9: User clicks on product to view details

Step 10: Product detail page shows full information

5.4 Add to Cart Process

Step 1: User views a product detail page

Step 2: User selects quantity and clicks "Add to Cart"

Step 3: Redux action dispatched with product ID and quantity

Step 4: Product details fetched from API if not in state

Step 5: Cart item created with product info and quantity

Step 6: Redux cart state updated with new item

Step 7: Cart data saved to localStorage for persistence

Step 8: Cart icon updates to show item count

Step 9: User can continue shopping or proceed to checkout

5.5 Checkout and Payment Process

Step 1: User clicks "Proceed to Checkout" from cart

Step 2: If not logged in, redirect to login page

Step 3: User enters shipping address details

Step 4: User selects payment method (Stripe/COD)

Step 5: Order summary displayed with prices calculated

Step 6: If Stripe selected, Stripe checkout modal opens

Step 7: User enters card details in Stripe form

Step 8: Stripe processes payment and returns token

Step 9: Frontend sends order data to `/api/orders`

Step 10: Backend creates order document in database

Step 11: Payment processed via Stripe API

Step 12: Order confirmation displayed to user

Step 13: Cart is cleared from Redux state and localStorage

5.6 Order Management Process

Step 1: User navigates to "My Orders" page

Step 2: Frontend fetches user's orders from `/api/orders/myorders`

Step 3: Backend authenticates user via JWT token

Step 4: Database queried for orders matching user ID

Step 5: Orders returned sorted by date (newest first)

Step 6: User can view order details and status

Step 7: Admin can update order status (Processing/Shipped/Delivered)

Step 8: Status changes trigger database update

Step 9: User receives updated order information

5.7 Contact Form Submission

Step 1: User navigates to Contact page

Step 2: User fills out contact form (name, email, message, city)

Step 3: Frontend validates form fields

Step 4: POST request sent to `/contact` endpoint

Step 5: Backend creates new Contact document

Step 6: Contact saved to MongoDB database

Step 7: Success message displayed to user

Step 8: Admin can view contact submissions in database

6 API Endpoints

Method	Endpoint	Description	Auth
POST	/api/users/register	Register new user	No
POST	/api/users/login	User login	No
GET	/api/users/profile	Get user profile	Yes
GET	/api/products	Get all products	No
GET	/api/products/:id	Get product by ID	No
POST	/api/orders	Create new order	Yes
GET	/api/orders/myorders	Get user's orders	Yes
GET	/api/orders/:id	Get order by ID	Yes
POST	/payment	Process Stripe payment	No
POST	/contact	Submit contact form	No
POST	/api/upload	Upload image file	No

Table 8: API Endpoints Summary

7 System Requirements

7.1 Software Requirements

- Node.js (v14 or higher)
- npm (v6 or higher)
- MongoDB (v4.4 or higher) or MongoDB Atlas account
- Modern web browser (Chrome, Firefox, Edge)
- Stripe account for payment processing

7.2 Environment Variables

```
PORT=5000
NODE_ENV=development
MONGO_URI=your_mongodb_connection_string
SECRET_KEY=your_stripe_secret_key
JWT_SECRET=your_jwt_secret_key
```

8 Conclusion

The Pharmacy Management Application provides a complete e-commerce solution for online pharmacies. Built using modern web technologies (MERN stack), it offers a scalable and maintainable codebase. The application handles user authentication securely, manages product inventory, processes payments through Stripe, and tracks orders efficiently.